

Lý thuyết chuyên sâu: Sàng số nguyên tố & Thừa số nguyên tố

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Cơ sở Số học về Số nguyên tố & Ước số

1.1. Định nghĩa & Định lý cơ bản của Số học (Fundamental Theorem of Arithmetic)

- Số nguyên tố (Prime number):** Là số tự nhiên lớn hơn 1 chỉ có đúng 2 ước dương là 1 và chính nó (2, 3, 5, 7, 11, ...). Số 2 là số nguyên tố chẵn duy nhất.
- Hợp số (Composite number):** Là số tự nhiên lớn hơn 1 có nhiều hơn 2 ước dương. (Lưu ý: Số 0 và 1 không phải là số nguyên tố, cũng không phải là hợp số).
- Định lý cơ bản của Số học:** Mọi số nguyên $n > 1$ đều phân tích được duy nhất (không xét đến thứ tự các thừa số) thành tích của các lũy thừa số nguyên tố:

$$n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k} \quad (p_1 < p_2 < \cdots < p_k, a_i \geq 1)$$

1.2. Các hàm số học dẫn xuất từ dạng phân tích thừa số nguyên tố

Từ dạng phân tích $n = \prod_{i=1}^k p_i^{a_i}$:

1. Số lượng các ước số dương $d(n)$ (hoặc $\tau(n)$):

Mỗi ước số d của n có dạng $d = p_1^{b_1} p_2^{b_2} \cdots p_k^{b_k}$ với $0 \leq b_i \leq a_i$. Theo quy tắc nhân:

$$d(n) = (a_1 + 1)(a_2 + 1) \cdots (a_k + 1) = \prod_{i=1}^k (a_i + 1)$$

2. Tổng các ước số dương $\sigma(n)$:

$$\sigma(n) = \prod_{i=1}^k (1 + p_i + p_i^2 + \cdots + p_i^{a_i}) = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

3. Tích tất cả các ước số dương $P(n)$:

Các ước số luôn ghép thành từng cặp $(d, n/d)$ có tích bằng n . Vì có $d(n)$ ước số nên:

$$P(n) = n^{\frac{d(n)}{2}}$$

4. **Hàm phi Euler** $\varphi(n)$: Số lượng số $1 \leq x \leq n$ thỏa mãn $\gcd(x, n) = 1$:

$$\varphi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right) = \prod_{i=1}^k p_i^{a_i-1} (p_i - 1)$$

2. Kiểm tra số nguyên tố đơn lẻ

2.1. Phương pháp thử chia (Trial Division) - $O(\sqrt{N})$

Bổ đề: Nếu n là hợp số, n phải có ít nhất một ước nguyên tố $p \leq \sqrt{n}$.

Chứng minh: Giả sử $n = a \cdot b$. Nếu cả $a > \sqrt{n}$ và $b > \sqrt{n}$ thì $a \cdot b > n$ (mâu thuẫn). Do đó, $\min(a, b) \leq \sqrt{n}$.

Tối ưu bước nhảy $6k \pm 1$:

Mọi số nguyên tố > 3 đều có dạng $6k \pm 1$ (vì các dạng $6k, 6k+2, 6k+4$ chia hết cho 2, còn $6k+3$ chia hết cho 3). Ta kiểm tra chia hết cho 2 và 3 trước, sau đó chỉ cần thử các ước dạng $6k \pm 1$ đến $\sqrt{n}$, giảm số phép thử xuống còn $1/3$.

2.2. Thuật toán kiểm tra số lớn Miller-Rabin Primality Test - $O(k \log^3 N)$

Dành cho N lớn (lên tới 10^{18}), khi $O(\sqrt{N}) \approx 10^9$ phép tính sẽ bị TLE.

- **Cơ sở lý thuyết:**
- Theo Định lý Fermat nhỏ: Nếu p nguyên tố và $\gcd(a, p) = 1$ thì $a^{p-1} \equiv 1 \pmod{p}$.
- Bổ đề căn bậc hai của 1: Phương trình $x^2 \equiv 1 \pmod{p}$ chỉ có đúng 2 nghiệm là $x \equiv 1$ hoặc $x \equiv -1 \pmod{p}$.
- **Thuật toán:**
Biểu diễn $n - 1 = 2^s \cdot d$ với d lẻ.
Với một cơ số a , ta tính dãy: $a^d, a^{2d}, a^{4d}, \dots, a^{2^{s-1}d}, a^{2^s d} \pmod{n}$.
Nếu $a^d \not\equiv 1 \pmod{n}$ và không có phần tử nào trong dãy đồng dư với $-1 \pmod{n}$, thì n chắc chắn là **hợp số**.
- **Tính tiền định (Deterministic) cho 64-bit:**
Đối với mọi số $n < 2^{64}$, chỉ cần thử tập cơ số gồm 7 số:

$$\mathcal{A} = \{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$$

(hoặc tập 12 số nguyên tố đầu tiên $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$) là **chính xác 100%**, không có bất kỳ sai số ngẫu nhiên nào!

3. Sàng nguyên tố Eratosthenes cổ điển (Classical Sieve)

3.1. Thuật toán & Các bước tối ưu

Tìm tất cả các số nguyên tố $\leq N$:

1. Khởi tạo mảng boolean `is_prime[0..N] = true`, đặt `is_prime[0] = is_prime[1] = false`.

2. Duyệt i từ 2 đến $\lfloor \sqrt{N} \rfloor$:

- Nếu i là nguyên tố, gạch bỏ tất cả các bội số của i bắt đầu từ i^2 với bước nhảy i :

$$i^2, i^2 + i, i^2 + 2i, \dots \leq N$$

(Lý do bắt đầu từ i^2 : Các bội nhỏ hơn dạng $k \cdot i$ với $k < i$ đã bị gạch bởi ước nhỏ hơn của k từ trước đó).

3.2. Đánh giá độ phức tạp thời gian: $O(N \ln(\ln N))$

Tổng số phép gạch bỏ:

$$S = \sum_{p \leq N, p \in \mathbb{P}} \frac{N}{p} = N \sum_{p \leq N} \frac{1}{p}$$

Theo **Định lý Mertens thứ hai (Mertens' Second Theorem)** trong giải tích số:

$$\sum_{p \leq N} \frac{1}{p} = \ln(\ln N) + M + O\left(\frac{1}{\ln N}\right)$$

(với $M \approx 0.261497$ là hằng số Meissel-Mertens).

Do đó, độ phức tạp thời gian là:

$$O(N \log \log N)$$

Với $N = 10^7$, $\log_2(\log_2(10^7)) \approx 4.5$, thời gian chạy thực tế tương đương $O(N)$ (khoảng 0.05s).

3.3. Tối ưu bộ nhớ với `std::bitset`

Mảng `bool is_prime[10000000]` tốn 10MB RAM (mỗi `bool` tốn 1 byte).

Dùng `std::bitset<10000001>` chỉ tốn **1 bit cho mỗi phần tử**:

$$\text{Memory} = \frac{10^7}{8 \times 1024 \times 1024} \approx 1.19\text{MB}$$

Tiết kiệm **87.5% bộ nhớ** và tận dụng tính năng cache line của CPU, giúp chương trình chạy nhanh hơn gấp 2-3 lần.

4. Sàng nguyên tố tuyến tính $O(N)$ (Linear Sieve / Euler's Sieve)

4.1. Hạn chế của Sàng Eratosthenes

Trong sàng Eratosthenes, nhiều hợp số bị gạch nhiều lần lặp đi lặp lại:

- 12 bị gạch bởi 2 và 3.
- 30 bị gạch bởi 2, 3 và 5.
- $2310 = 2 \times 3 \times 5 \times 7 \times 11$ bị gạch 5 lần.

4.2. Nguyên lý Sàng Euler $O(N)$

Mục tiêu: **Mỗi hợp số x chỉ bị gạch đúng một lần duy nhất bởi ước nguyên tố nhỏ nhất (Smallest Prime Factor - SPF) của nó.**

Mọi hợp số x đều biểu diễn được duy nhất dưới dạng:

$$x = i \cdot p \quad \text{với } p = \text{spf}[x] \text{ và } p \leq \text{spf}[i]$$

Thuật toán:

1. Duyệt i từ 2 đến N :

- Nếu $\text{spf}[i] == 0$, i là số nguyên tố $\rightarrow$ thêm vào danh sách `primes` và gán $\text{spf}[i] = i$.

2. Với mỗi số nguyên tố $p \in \text{primes}$:

- Nếu $p > \text{spf}[i]$ hoặc $i \cdot p > N \implies$ **Dừng vòng lặp (Break).**
- Gán $\text{spf}[i \cdot p] = p$.

🔑 **Điểm mấu chốt:** Khi $i : p$ (tức $\text{spf}[i] == p$), ta phải dừng ngay lập tức.

Nếu tiếp tục nhân với số nguyên tố kế tiếp $p' > p$, thì số $y = i \cdot p'$ sẽ có ước nguyên tố nhỏ nhất là p (chứ không phải p'), và y sẽ bị gạch lần thứ hai khi vòng ngoài xét đến $i' = y/p$. Lệnh `break` đảm bảo mỗi hợp số chỉ được gạch đúng 1 lần!

Vì mỗi số từ 2 đến N chỉ bị gán `spf` đúng 1 lần, độ phức tạp thời gian đạt **chính xác** $O(N)$.

5. Ứng dụng đột phá của mảng SPF: Phân tích thừa số nguyên tố trong $O(\log N)$

Sau khi tiền xử lý mảng `spf` trong $O(N)$, ta có thể phân tích bất kỳ số nguyên $X \leq N$ ra thừa số nguyên tố trong thời gian tỷ lệ với số lượng ước nguyên tố, tức $O(\log X)$:

```
vector<pair<int, int>> factorize(int x) {
    vector<pair<int, int>> factors;
    while (x > 1) {
        int p = spf[x];
        int count = 0;
        while (x % p == 0) {
            count++;
            x /= p;
        }
        factors.push_back({p, count});
    }
    return factors;
}
```

Với $Q = 10^6$ truy vấn phân tích số:

- Phương pháp thử chia tốn $O(Q\sqrt{N}) \approx 10^6 \times 3162 \approx 3 \times 10^9$ phép tính $\rightarrow$ **TLE**.
- Phương pháp dùng `spf` tốn $O(N + Q \log N) \approx 10^7 + 10^6 \times 20 \approx 3 \times 10^7$ phép tính $\rightarrow$ **AC mượt mà trong 0.2s**.

6. Sàng trên đoạn $[L, R]$ (Segmented Sieve)

6.1. Bài toán đặt ra

Tìm tất cả các số nguyên tố trong đoạn $[L, R]$ với:

$$1 \leq L \leq R \leq 10^{12} \quad \text{và} \quad R - L \leq 10^7$$

Ta không thể tạo mảng kích thước 10^{12} để sàng (quá bộ nhớ).

6.2. Giải pháp: Sàng phân đoạn

1. Theo bổ đề, mọi hợp số $x \in [L, R]$ đều có ít nhất một ước nguyên tố $p \leq \sqrt{R} \leq \sqrt{10^{12}} = 10^6$.
2. Dùng sàng Eratosthenes chuẩn để tìm tất cả các số nguyên tố $\leq \sqrt{R}$.
3. Tạo mảng boolean đánh dấu kích thước nhỏ $K = R - L + 1 \leq 10^7$.
4. Với mỗi số nguyên tố $p \leq \sqrt{R}$:
5. Tìm bội số nhỏ nhất của p mà $\geq L$:

$$\text{start} = \max \left(p^2, \left\lceil \frac{L}{p} \right\rceil \cdot p \right) = \max \left(p^2, \frac{L + p - 1}{p} \cdot p \right)$$

6. Gạch bỏ tất cả các bội $\text{start}, \text{start} + p, \text{start} + 2p, \dots \leq R$.
7. Khi gạch, tịnh tiến chỉ số về mảng nhỏ: `is_prime[x - L] = false`.

Độ phức tạp:

- Thời gian: $O(\sqrt{R} \log \log \sqrt{R} + (R - L + 1) \log \log \sqrt{R})$.
- Bộ nhớ: $O(\sqrt{R} + (R - L + 1))$.

7. Tài liệu tham khảo & Bài giảng đề xuất (References)

7.1. Giáo trình & Sách thuật toán kinh điển

1. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
2. [Chapter 21: Primes and Factorization \(Sieve of Eratosthenes, Linear Sieve, Primality Tests\)](#).
3. **Introduction to Algorithms (CLRS 4th Edition)** — *Thomas H. Cormen et al.*:
4. *Chapter 31: Number-Theoretic Algorithms (Section 31.8: Primality testing & Miller-Rabin)*.
5. **Elementary Number Theory (7th Edition)** — *David M. Burton*:
6. *Chapter 3: Primes and Their Distribution (The Sieve of Eratosthenes, The Prime Number Theorem)*.
7. **Competitive Programming 4 (CP4 - Book 1)** — *Steven Halim*:
8. *Section 5.3: Finding Primes in $O(N)$ and Factorization*.

7.2. Bài giảng & Chuyên đề trực tuyến (Online Resources & Wikis)

1. **VNOI Wiki**:
2. [Số nguyên tố \(Primes\)](#) — Cẩm nang toàn diện về số nguyên tố và các thuật toán sàng.
3. [Kiểm tra số nguyên tố Miller-Rabin](#) — Chi tiết cài đặt thuật toán Miller-Rabin $O(\log N)$.
4. [Thuật toán Pollard's Rho](#) — Phân tích thừa số nguyên tố cho số 64-bit trong $O(N^{1/4})$.
5. **CP-Algorithms**:
6. [Sieve of Eratosthenes](#) — Sàng cổ điển, tối ưu bộ nhớ bitset và phân đoạn.
7. [Linear Sieve](#) — Sàng tuyến tính Euler và tính hàm nhân tính.
8. [Primality Tests \(Miller-Rabin\)](#).
9. [Integer Factorization](#).



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.